

以太坊（3）——权益证明与智能合约

智能合约

首先明确一下几个说法（说法不严谨，为了介绍清晰才说的）：

- 1. 全节点==矿工
- 2. 节点==账户

智能合约是基于Solidity语言编写的

WTF

Solidity ▾ Ethers ▾ EVM ▾ Layer 2 ▾ Frontend ▾ zk ▾ AI ▾ Basecamp ▾

中文

看板

1. HelloWeb3 (三行代码)

2. 数值类型

3. 函数类型

4. 函数输出

5. 变量数据存储和作用域

6. 引用类型

7. 映射类型

8. 变量初始值

9. 常数

10. 控制流

11. 构造函数和修饰器

12. 事件

13. 继承

14. 抽象合约和接口

15. 异常

映射类型

进度 100% 状态 完成

变量初始值

进度 100% 状态 完成

常数

进度 100% 状态 完成

控制流

进度 100% 状态 完成

构造函数和修饰器

进度 100% 状态 完成

事件

进度 100% 状态 完成

继承

进度 100% 状态 完成

抽象合约和接口

进度 100% 状态 完成

异常

进度 100% 状态 完成

学习Solidity基础内容，开始智能合约之旅！基于WTF Solidity极简教程前15讲。

学习Solidity语言可以到WTF学院官网（[Hello from WTF Academy | WTF Academy](#)），有初级和进阶课程

智能合约的结构

下面的函数有一些问题，在文章最后修正

```
pragma solidity ^0.4.21;

contract SimpleAuction {
    //...address public beneficiary;...//拍卖受益人
    //...uint public auctionEnd;...//结束时间
    //...address public highestBidder;...//当前的最高出价人
    //...mapping(address => uint) bids;...//所有竞拍者的出价
    //...address[] bidders;...//所有竞拍者

    //...// 需要记录的事件
    //...event HighestBidIncreased(address bidder, uint amount);
    //...event Pay2Beneficiary(address winner, uint amount);

    //...// 以受益者地址`_beneficiary`的名义,
    //...// 创建一个简单的拍卖, 拍卖时间为`_biddingTime`秒。
    //...constructor(uint _biddingTime, address _beneficiary
    //...    ) public {
    //...    beneficiary = _beneficiary;
    //...    auctionEnd = now + _biddingTime;
    //...}

    //...// 对拍卖进行出价, 随交易一起发送的ether与之前已经发送的
    //...// ether的和为本次出价。
    //...function bid() public payable { ...
    //...}

    //...// 使用withdraw模式
    //...// 由投标者自己取回出价, 返回是否成功
    //...function withdraw() public returns (bool) { ...
    //...}

    //...// 结束拍卖, 把最高的出价发送给受益人
    //...function pay2Beneficiary() public returns (bool) { ...
    //...}
}
```

声明使用solidity的版本

状态变量

log记录

构造函数, 仅在合约创建时调用一次

成员函数, 可以被一个外部账户或合约账户调用

本实例改编自Solidity文档: 简单的公开拍卖

```
// 对拍卖进行出价
// 随交易一起发送的ether与之前已经发送的ether的和为本次出价
function bid() public payable {
    // 对于能接收以太币的函数, 关键字 payable 是必须的。

    // 拍卖尚未结束
    require(now <= auctionEnd);
    // 如果出价不够高, 本次出价无效, 直接报错返回
    require(bids[msg.sender]+msg.value > bids[highestBidder]);

    // 如果此人之前未出价, 则加入到竞拍者列表中
    if (!(bids[msg.sender] == uint(0))) {
        bidders.push(msg.sender);
    }
    // 本次出价比当前最高价高, 取代之
    highestBidder = msg.sender;
    bids[msg.sender] += msg.value;
    emit HighestBidIncreased(msg.sender, bids[msg.sender]);
}
```

```
// 结束拍卖, 把最高的出价发送给受益人,
// 并把未中标的出价者的钱返还
function auctionEnd() public {
    // 拍卖已截止
    require(now > auctionEnd);
    // 该函数未被调用过
    require(!ended);

    // 把最高的出价发送给受益人
    beneficiary.transfer(bids[highestBidder]);
    for (uint i = 0; i < bidders.length; i++) {
        address bidder = bidders[i];
        if (bidder == highestBidder) continue;
        bidder.transfer(bids[bidder]);
    }

    ended = true;
    emit AuctionEnded(highestBidder, bids[highestBidder]);
}
```

上述是一个简单的智能合约的结构:

1. 一个合约可以看作是一个类
2. log通过emit触发事件在链上记录下来
3. 构造函数是每个合约有且只有一个的, 可以为空, 只能在创建时调用一次
4. 成员函数是部署合约后可以调用的, 一般有关键字, modifier等限制权限
5. 智能合约里的mapping不能遍历, 只能存储它的地址集合, 按照地址去查询

合约的调用

外部账户的调用

BACK

TX 0x73275297b391f3e08b1cc7144d7ab5fcf77fecee92b46ca9ec2946f56ebf8ea2

SENDER ADDRESS	TO CONTRACT ADDRESS	CONTRACT CALL		
0x903db0EbD420669Ab50BCF93c550df9b5Da178c	0x5E31d519A6F34d224C25B706687EE2AbF170B888			
VALUE	GAS USED	GAS PRICE	GAS LIMIT	MINED IN BLOCK
0.00 ETH	21657	1000000000	6000000	3
TX DATA	0x2a24f46c			

1. 首先, 要填写外部账户的地址, 合约账户的地址

2. 之后，需要填写函数接收的参数（之后细说）

3. 一般部署和调用合约的编译器是Remix

一个合约中调用另一个合约的函数

直接调用

1. 直接调用

```
3 contract A {
4     event LogCallFoo(string str);
5     function foo(string str) returns (uint){
6         emit LogCallFoo(str);
7         return 123;
8     }
9 }
10
11 contract B {
12     uint ua;
13     function callAFooDirectly(address addr) public{
14         A a = A(addr);
15         ua = a.foo("call foo directly");
16     }
17 }
```

- 如果在执行a.foo()过程中抛出错误，则callAFooDirectly也抛出错误，本次调用全部回滚。
- ua为执行a.foo("call foo directly")的返回值
- 可以通过.gas() 和 .value() 调整提供的gas数量或提供一些ETH

1. 合约B调用合约A的内容，`A a = A(addr)`,addr指的是A的地址，该语句将这个地址转换成A合约的实例a，之后调用a里面的函数foo，使用ua接收返回值。
2. 由于以太坊规定一个交易只能由外部账户调用，合约账户不能发起交易。因此只能当某个外部账户调用合约B的时候，才能触发B中的那个语句，进而调用A
3. 这种调用方式下，如果A在运行过程中发生什么异常，会导致B合约的内容也连带回滚

call函数调用

使用address类型的call()函数

```
contract C {
    function callAFooByCall(address addr) public returns (bool){
        bytes4 funcsig = bytes4(keccak256("foo(string)"));
        if (addr.call(funcsig,"call foo by func call"))
            return true;
        return false;
    }
}
```

- 第一个参数被编码成4 个字节，表示要调用的函数的签名。
- 其它参数会被扩展到 32 字节，表示要调用函数的参数。
- 上面的这个例子相当于[A\(addr\).foo\("call foo by func call"\)](#)
- 返回一个布尔值表明了被调用的函数已经执行完毕（true）或者引发了一个 EVM 异常（false），无法获取函数返回值。
- 也可以通过.gas() 和 .value() 调整提供的gas数量或提供一些ETH

这种方式的调用，如果调用的合约（addr的实例）出现错误，C合约不会回滚，只会引起调用部分返回异常值，其余部分正常

代理调用 delegatecall()

- 使用方法与call()相同，只是不能使用.value()
- 区别在于是否切换上下文
 - call()切换到被调用的智能合约上下文中
 - delegatecall()只使用给定地址的代码，其它属性（存储，余额等）都取自当前合约。delegatecall 的目的是使用存储在另外一个合约中的库代码。

这种调用不需要切换到被调用的合约环境中去执行，在当前合约的环境中执行（使用当前合约的属性等）

1.

fallback函数

注意，只有向合约账户转账时才有fallback函数的生成！！

fallback()函数

```
function() public [payable]{  
    .....  
}
```

- 匿名函数，没有参数也没有返回值。
- 在两种情况下会被调用：
 - 直接向一个合约地址转账而不加任何data
 - 被调用的函数不存在
- 如果转账金额不是0，同样需要声明payable，否则会抛出异常。

如果不需要调用函数或者调用函数错误时，Solidity编译器自动生成fallback函数，里面的payable函数也是自动添加的

注意：gas费是给记录你这笔交易矿工的（类似于BTC里面的交易手续费），不是给合约账户的！

合约节点会在以下情况下调用fallback函数：

1. **未知函数调用：**当合约接收到一个未知的函数调用时，即调用合约中不存在的函数时，会触发fallback函数的执行。
2. **以太币转账：**当以太币被直接发送到合约地址（即没有附带调用数据），这时会触发fallback函数的执行。
3. **低级调用：**如果合约内部使用了 `call` 或 `delegatecall` 调用其他合约，并且调用的合约不存在对应的函数时，也会调用fallback函数。

错误处理

智能合约的调用具有原子性，即一个合约部分出错，整体回滚（这个和前面讲的addr调用不冲突，addr调用是使用一个判断语句将这个调用变成了一个返回值）

1. gaslimit被耗完依旧不能实现，调用被回滚，而且gas费不退还
2. assert()语句检查内部错误（类似于C语言）
3. require()语句检查外部错误（比如msg.sender!=addr）
4. revert()语句自动弹出错误（可用于if条件等）

注意： Solidity里面**没有**自定义报错类型try-catch

嵌套调用

一个合约调用另一个合约

如果向某个合约账户转账，虽然表面上你没有调用，但是编译器自动帮你生成了fallback函数，还是调用了！

智能合约的创建和运行

创建

1. 外部账户发起转账到0x0地址中，转账金额为0
2. 合约代码（编译成bytecode）放在data域中
3. 支付给矿工gas，矿工将合约发布到区块链上，返回合约的地址（也就是智能合约的地址）
4. 此时，智能合约就可以被所有人调用了

运行

智能合约运行在**以太坊虚拟机EVM**（Ethereum Virtual Machine）上

EVM是一个256位的WWC（Worldwide Computer），使用EVM提高了智能合约的可移植性

汽油费（gas）

原因

图灵完备的语言，需要避免死循环导致的停机

数据结构

调用合约的人支付，相当于是把停机问题推给了调用方

```
type txdata struct {
    AccountNonce uint64 `json:"nonce" gencodec:"required"`
    Price         *big.Int `json:"gasPrice" gencodec:"required"`
    GasLimit      uint64 `json:"gas" gencodec:"required"`
    Recipient     *common.Address `json:"to" rlp:"nil" // nil means contract creation`
    Amount        *big.Int `json:"value" gencodec:"required"`
    Payload       []byte `json:"input" gencodec:"required"`
}
```

GasLimit就是调用方此次调用这个合约能接受的最大汽油量，可以在Remix部署时填写

GasLimit * Price 计算出的是花掉的最大汽油费

Payload就是data域

gas的价格是由调用者选择的，价格高的更容易被矿工写入区块。

1. 往往代码功能简单的汽油费低，代码复杂的汽油费高。
2. 存储状态变量汽油费高，仅读取状态变量免费

调用过程

1. 当一个全节点收到一个调用的时候，它首先计算出调用花费的最大汽油费
2. 将这笔最大汽油费从发起账户中扣掉。
3. 然后根据实际执行的情况算出实际花费的汽油费，剩下的退回去，不够的话合约回滚gas耗完不退

数据结构

区块头

GasLimit

实际消耗gas的上限！

BTC每个区块最大不能超过1M，主要是带宽因素

1. 由于ETH矿工的gas费收益是很可观的，防止他为了多收钱把巨量的交易包含进去，影响传输速率，也要进行限制，
2. 但是因为智能合约Solidity的结构复杂（循环和调用），往往大小不能完全反映复杂程度，因此就使用最大gas费的方式限制

GasUsed

实际消耗的汽油费

每个区块发布的时候，矿工可以对gaslimit进行微调，上下不超过1/1024

三树

合约调用的时间应该在发布区块前还是发布区块后？

先执行，后挖矿。原因是执行完合约之后，“三树”的信息会改变，而区块头包含的是三树的根哈希值。只有得到根哈希值之后才可以组合数据，尝试nonce值！

本质上，汽油费是为了补偿矿工执行合约消耗的资源。但是对于没有挖到区块的矿工，他们的验证没有补偿的，这会不会导致那些矿工不去验证发布的区块，就默认发布的是对的。如果这样的话就会严重影响区块链的安全性！

节点不验证了怎么办？

如果它不执行不验证，就无法更新三树，之后它组装区块的时候算出的根哈希值是错的，挖出区块的nonce值被其他区块验证为不合法！环环相扣了属于是。

我还是不验证，直接抄！

这种做法类似于矿池的做法，一个全节点进行验证，其他矿工只负责计算哈希值。

但是如果你不是这个矿池的成员，你敢不敢相信它发布的三树根哈希是正确的呢？要知道挖矿的3Ether收益可是远远大于gas费的，矿工不愿意冒着辛辛苦苦挖出区块作废的奉劝去节省验证的花销。

执行错误的交易要不要发布？

要，这样依旧可以扣除他的汽油费，获得你的收益

智能合约是否支持多线程（多核并行处理）？

不支持，因为Solidity语言中不具有支持多线程的语句。原因是多核对于内存访问顺序不同的时候，会导致最终的结果不同，而以太坊需要的是状态机进入**统一的确定的状态** ([Papers\(zhenxiao.com\)](http://Papers(zhenxiao.com)))可以参考肖老师的论文

Receipt

```
45 // Receipt represents the results of a transaction.
46 type Receipt struct {
47     // Consensus fields
48     PostState      []byte `json:"root"`
49     Status         uint64 `json:"status"`
50     CumulativeGasUsed uint64 `json:"cumulativeGasUsed" gencodec:"required"`
51     Bloom          Bloom   `json:"logsBloom"           gencodec:"required"`
52     Logs           []*Log `json:"logs"                gencodec:"required"`
53
54     // Implementation fields (don't reorder!)
55     TxHash          common.Hash `json:"transactionHash" gencodec:"required"`
56     ContractAddress common.Address `json:"contractAddress"`
57     GasUsed          uint64      `json:"gasUsed" gencodec:"required"`
58 }
```

每个交易执行完形成一个收据（receipt），Status这个变量代表的是交易的状态

地址类型

`<address>.balance (uint256):`

以 Wei 为单位的 **地址类型** 的余额。

`<address>.transfer(uint256 amount):`

向 **地址类型** 发送数量为 amount 的 Wei，失败时抛出异常，发送 2300 gas 的矿工费，不可调节。

`<address>.send(uint256 amount) returns (bool):`

向 **地址类型** 发送数量为 amount 的 Wei，失败时返回 `false`，发送 2300 gas 的矿工费用，不可调节。

`<address>.call(...) returns (bool):`

发出底层 `CALL`，失败时返回 `false`，发送所有可用 gas，不可调节。

`<address>.callcode(...) returns (bool):`

发出底层 `CALLCODE`，失败时返回 `false`，发送所有可用 gas，不可调节。

`<address>.delegatecall(...) returns (bool):`

发出底层 `DELEGATECALL`，失败时返回 `false`，发送所有可用 gas，不可调节。

address.balance: address的余额

address.transfer(金额):当前合约向address转入的金额

address.call:当前合约调用address

转账的方法

```
<address>.transfer(uint256 amount)//会导致连锁型回滚 gas少  
<address>.send(uint256 amount)//返回false 不会会滚 gas少  
<address>.call.value(uint256 amount)//本意不是专门为转账设计的 不会会滚 发送剩余的所有gas
```

智能合约的信息获取

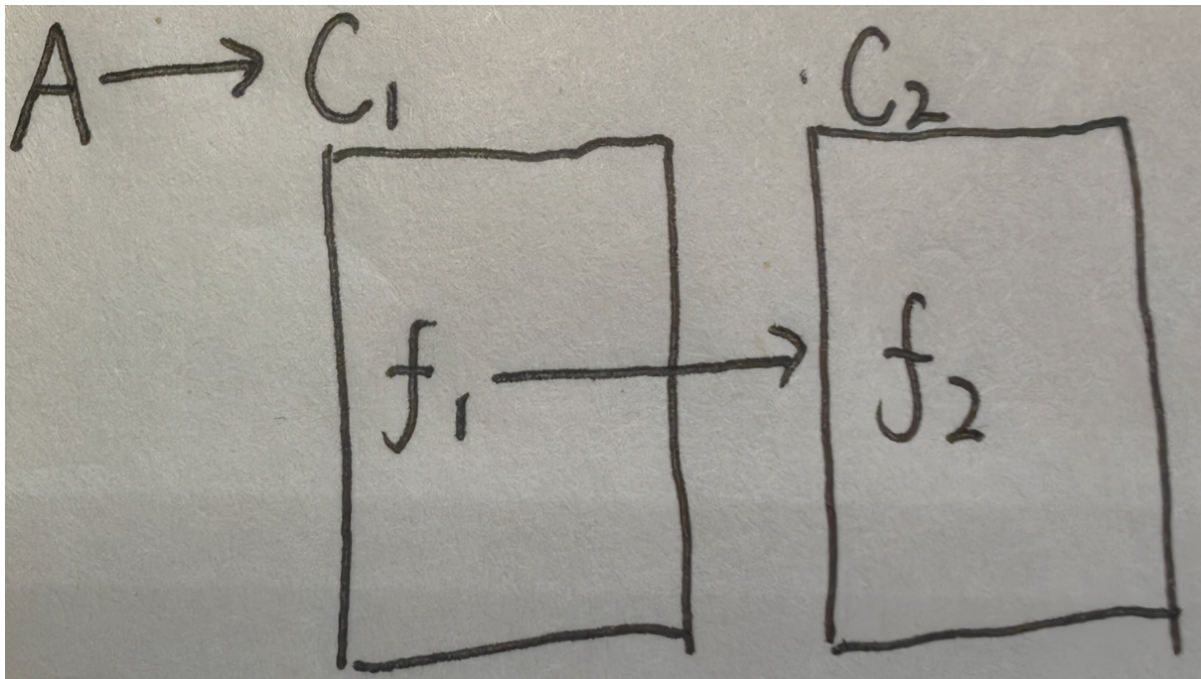
通过智能合约可以得到区块链的信息

- `block.blockhash(uint blockNumber) returns (bytes32)`: 给定区块的哈希—仅对最近的 256 个区块有效而不包括当前区块
- `block.coinbase ( address )`: 挖出当前区块的矿工地址
- `block.difficulty ( uint )`: 当前区块难度
- `block.gaslimit ( uint )`: 当前区块 gas 限额
- `block.number ( uint )`: 当前区块号
- `block.timestamp ( uint )`: 自 unix epoch 起始当前区块以秒计的时间戳

智能合约可以获取的调用信息

- `msg.data ( bytes )`: 完整的 calldata
- `msg.gas ( uint )`: 剩余 gas
- `msg.sender ( address )`: 消息发送者（当前调用）
- `msg.sig ( bytes4 )`: calldata 的前 4 字节（也就是函数标识符）
- `msg.value ( uint )`: 随消息发送的 wei 的数量
- `now ( uint )`: 目前区块时间戳（`block.timestamp`）
- `tx.gasprice ( uint )`: 交易的 gas 价格
- `tx.origin ( address )`: 交易发起者（完全的调用链）

msg.sender和tx.origin的区别



上图中A账户调用C1合约中的f1函数，f1函数调用C2合约中的f2函数

则对于f2函数而言，msg.sender指的是C1，而tx.origin指的是A账户

智能合约的设计

不可篡改!

智能合约是不可篡改的，在发布之前一定要反复测试。

一旦发布就不可撤销，无法更改。可能会导致资金的永久锁死或者被黑客利用漏洞进行攻击!

测试的网址比如本地的truffle, ganache, 测试网等等，确认完全没有问题再进行发布

后门?

不存在的，因为这与去中心化背道而驰

拍卖函数的修正

第一版问题

```
/// 对拍卖进行出价
/// 随交易一起发送的ether与之前已经发送的ether的和为本次出价
function bid() public payable {
    // 对于能接收以太币的函数，关键字 payable 是必须的。

    // 拍卖尚未结束
    require(now <= auctionEnd);
    // 如果出价不够高，本次出价无效，直接报错返回
    require(bids[msg.sender]+msg.value > bids[highestBidder]);

    //如果此人之前未出价，则加入到竞拍者列表中
    if (!bids[msg.sender] == uint(0)) {
        bidders.push(msg.sender);
    }
    //本次出价比当前最高价高，取代之
    highestBidder = msg.sender;
    bids[msg.sender] += msg.value;
    emit HighestBidIncreased(msg.sender, bids[msg.sender]);
}
```

```
/// 结束拍卖，把最高的出价发送给受益人，
/// 并把未中标的出价者的钱返还
function auctionEnd() public {
    //拍卖已截止
    require(now > auctionEnd);
    //该函数未被调用过
    require(!ended);

    //把最高的出价发送给受益人
    beneficiary.transfer(bids[highestBidder]);
    for (uint i = 0; i<bidders.length;i++){
        address bidder = bidders[i];
        if (bidder == highestBidder) continue;
        bidder.transfer(bids[bidder]);
    }

    ended = true;
    emit AuctionEnded(highestBidder, bids[highestBidder]);
}
```

都拿不到钱!

第二版

```
36    /// 使用withdraw模式
37    /// 由投标者自己取回出价，返回是否成功
38    function withdraw() public returns (bool) {
39        // 拍卖已截止
40        require(now > auctionEnd);
41        // 竞拍成功者需要把钱给受益人，不可取回出价
42        require(msg.sender != highestBidder);
43        // 当前地址有钱可取
44        require(bids[msg.sender] > 0);
45
46        uint amount = bids[msg.sender];
47        if (msg.sender.call.value(amount)()) {
48            bids[msg.sender] = 0;
49            return true;
50        }
51        return false;
52    }
```

拆成两个函数，第一个允许竞拍的失败者取回自己的钱（只能取一次），第二个是将最高出价给卖家

BUG

重入攻击（Re-entrancy Attack）

- 当合约账户收到ETH但未调用函数时，会立刻执行fallback()函数
- 通过addr.send()、addr.transfer()、addr.call.value()()三种方式付钱都会触发addr里的fallback函数。
- fallback()函数由用户自己编写

```
36    /// 使用withdraw模式
37    /// 由投标者自己取回出价，返回是否成功
38    function withdraw() public returns (bool) {
39        // 拍卖已截止
40        require(now > auctionEnd);
41        // 竞拍成功者需要把钱给受益人，不可取回出价
42        require(msg.sender != highestBidder);
43        // 当前地址有钱可取
44        require(bids[msg.sender] > 0);
45
46        uint amount = bids[msg.sender];
47        if (msg.sender.call.value(amount)()) {
48            bids[msg.sender] = 0;
49            return true;
50        }
51        return false;
52    }
```

```
pragma solidity ^0.4.21;
import "./SimpleAuctionV2.sol";

contract HackV2 {
    uint stack = 0;

    function hack_bid(address addr) payable public {
        SimpleAuctionV2 sa = SimpleAuctionV2(addr);
        sa.bid.value(msg.value)();
    }

    function hack_withdraw(address addr) payable {
        SimpleAuctionV2(addr).withdraw();
    }

    function() payable {
        stack += 2;
        if (msg.sender.balance >= msg.value && msg.gas > 6000 && stack < 500){
            SimpleAuctionV2(msg.sender).withdraw();
        }
    }
}
```

注意黑客合约的fallback函数（右侧最下面），当合约通过call函数调用时，调用fallback函数（忘记的话可以回顾一下上面的fallback函数部分）。因此，主合约的withdraw进入if语句的判断时，就会调用fallback函数给黑客转账，而fallback函数再次调用withdraw函数，形成循环（根本无法执行48行的清零语句）...

循环截至的条件：

1. 合约存储的钱不足以支付下次转账
2. 调用栈溢出
3. gas费不足

这也是黑客的fallback函数里面if语句判断的条件！

第三版

```
54     event Pay2Beneficiary(address winner, uint amount);
55     /// 结束拍卖，把最高的出价发送给受益人
56     function pay2Beneficiary() public returns (bool) {
57         // 拍卖已截止
58         require(now > auctionEnd);
59         // 有钱可以支付
60         require(bids[highestBidder] > 0);
61
62         uint amount = bids[highestBidder];
63         bids[highestBidder] = 0;
64         emit Pay2Beneficiary(highestBidder, bids[highestBidder]);
65
66         if (!beneficiary.call.value(amount)()) {
67             bids[highestBidder] = amount;
68             return false;
69         }
70         return true;
71     }
```

先清零再转账，这遵循了和其他合约发生交互的编程模式：

先判断条件，再改变条件，最后交互！

第二版纠正

另一种方法就是不用call！

```
require(bids[msg.sender] < 0);
```

```
uint amount = bids[msg.sender];
if (msg.sender.call.value(amount)()) {
    bids[msg.sender] = 0;
    return true;
}
return false;
```

```
uint amount = bids[msg.sender];
bids[msg.sender] = 0;
if (!msg.sender.send(amount)) {
    bids[msg.sender] = amount;
    return true;
}
return false;
```

使用send和transfer都可以，因为发送出的汽油费只有2300个单位，不足以支撑再一次的调用，只够写一个log

The DAO

造成以太坊分裂的一次攻击

DAO: Decentralized Autonomous Organization 去中心化机构

致力于去中心化投资的公司——The DAO

运行原理

这个公司本质上是ETH上的一个智能合约，你想要投资就要把你的以太币发给这个合约，换取合约的D代币（简称）。要投资什么项目由大家投票决定，投票的权重按照所拥有的D代币分配，收益也是按照规定按比例分配。

取款

取款的方式使用split DAO。当取钱时，收回相应的代币，将对应的以太币打到子基金Chile DAO之中（这种做法也是给部分人投资小众项目拆分子基金用的，极端例子就是单个投资者成立子基金，也就是取钱）

拆分之前有7天的讨论期，用于商讨要不要拆出子基金以及要不要加入这个子基金

拆分后有28天的锁定期，子基金里的钱要28天以后才能取出来

历史事件

2016年开始众筹，引起很大关注，在一个月时间筹集到价值1.5亿美元的Ether（当时价格）

```
function splitDAO(
    uint _proposalID,
    address _newCurator
) noEther onlyTokenholders returns (bool _success) {
    .....
    // Burn DAO Tokens
    Transfer(msg.sender, 0, balances[msg.sender]);
    withdrawRewardFor(msg.sender); // be nice, and get his rewards
     totalSupply -= balances[msg.sender];
    balances[msg.sender] = 0;
    paidOut[msg.sender] = 0;
    return true;
}
```

代码来源：

<https://etherscan.io/address/0x304a554a310C7e546dfe434669C62820b7D83490#code>

这就违反了之前提到的**先判断条件，再改变条件，最后交互！**的编程原则，导致了黑客盗走价值5kw美元的资金

之后，社区发生意见的分歧，一方认为要回滚，利用28天的锁定期来防止黑客取走资金。另一方认为，合约的漏洞只是它自己的漏洞，不能因为一个合约的问题就违背以太坊的不可篡改性。

社区的开发者认为The DAO是too big to fail，垮掉会对以太坊造成毁灭性打击（以太币价格跳水），因此采取了回滚措施进行补救。

如何补救？

首先，以太坊开发团队想到的是一个软分叉方案：

锁定黑客账户：进行一次软件升级但凡与TheDAO相关的账户，不能进行任何交易（老认新，软分叉）。

问题是，当时规定，这些“非法”的账户发布交易，不能上链也不收取gas，因此引起了很多这种账户发布死循环攻击来骚扰矿工，矿工不堪重负，纷纷回滚这次软件的升级

由于软方案失败，所剩时间不多，团队只能采取**硬分叉方案：**

将所有资金（要是只转黑客的，其他的相关账户都可以利用这个BUG实现攻击）转入一个账户B，该账户的唯一作用就是——退钱！在第192W个区块强制将所有相关账户的钱转给账户B（新认老，固执节点不认可你没有人家账户签名就转账的操作）

分歧

很多节点认为，这种操作违背了去中心化的理念（以太坊团队说转走谁的钱就转走）因此以太坊团队发布两个合约进行投票，支持哪一种方案就把以太币投进那个合约。最后大多节点（按掌握资金数量分）选择硬分叉。

但是，另一方不认可这样的做法：

1. 并非所有资金都参与投票了
2. 大多数人支持的就一定对吗？

这些矿工挖出的旧链以太币被称为ETC

ETC

ETC (classic) ，指的是在旧链上挖矿产出的以太币，由于挖矿算力大幅削减，不乏投机者选择在这条链上进行挖矿，但是也有一部分矿工坚持挖矿是源自去中心化的信仰！尽管开始的前景并不被人看好，但是这条链依旧坚持至今。



2015年7月 Vitalik Buterin和以太坊基金会创建了第一个基于区块链的图灵完成智能合约平台。

2016年4月~6月 以太坊The DAO项目ICO以遭到黑客攻击告终

2016年7月20日 以太坊硬分叉实施，产生了ETH和ETC两条独立的区块链，ETC正式诞生

2016年8月 91， pool上线

2016年9月23日，微软加入对于ETC的支持

2016年10月17日，ETC第一个ICO项目ETCWIN

2016年10月，ETCFans上线

2016年12月24日，全球第一个以太坊原链社区交易所ETCWin上线

2017年4月27日，ETF基金会确定限产

2017年6月底，ETC发起支持零知识证明改进建议。

.....

为了更好的管理这两条链，产生了新的数据标记——ChainID

Beauty Chain（美链）

背景介绍

美链(Beauty Chain)是一个部署在以太坊上的智能合约，有自己的代币BEC。

1. ICO: Initial Coin Offering没有自己的区块链，代币的发行、转账都是通过调用智能合约中的函数来完成的
2. 可以自己定义发行规则，每个账户有多少代币也是保存在智能合约的状态变量里
3. ERC 20 (Ethereum Request for Comments) 是以太坊上发行代币的一个标准，规范了所有发行代币的合约应该实现的功能和遵循的接口
4. 美链中有一个叫batchTransfer的函数，它的功能是向多个接收者发送代币，然后把这些代币从调用者的帐户上扣除

很多代币上线之前是依附在以太坊上的。

实现

```
function batchTransfer(address[] _receivers, uint256 _value) public whenNotPaused returns (bool) {
    uint cnt = _receivers.length;
    uint256 amount = uint256(cnt) * _value;
    require(cnt > 0 && cnt <= 20);
    require(_value > 0 && balances[msg.sender] >= amount);

    balances[msg.sender] = balances[msg.sender].sub(amount);
    for (uint i = 0; i < cnt; i++) {
        balances[_receivers[i]] = balances[_receivers[i]].add(_value);
        Transfer(msg.sender, _receivers[i], _value);
    }
    return true;
}
```

来源: <https://etherscan.io/address/0xc5d105e63711398af9bbff092d4b6769c82f793d#code>

接收者的数目最多20个

问题

```
uint amount = uint256(cnt) * _value
```

乘出的结果可能过大，出现溢出，即扣除的代币数字很小。也就是调用者转出的代币少，接收者依旧收到海量代币。也就是系统发行代币量凭空增多。

攻击细节

[illegible]

- 第0号参数是_receivers数组在参数列表中的位置，即从第64个byte开始，也就是第2号参数
 - 第2号参数先指明数组长度为2，然后第3号参数和第4号参数表明两个接受者的地址
- 第1号参数是给每个接受者转账的金额
- 通过这样的参数计算出来的amount恰好溢出为0!

来源: <https://etherscan.io/tx/0xad89ff16fd1ebe3a0a7cf4ed282302c06626c1af33221ebe0d3a470aba4a660f>

[illegible]

来源: <https://etherscan.io/tx/0xad89ff16fd1ebe3a0a7cf4ed282302c06626c1af33221ebe0d3a470aba4a660f>

每个黑客区块收到了很大一部分代币

反思

Is smart contract real smart?

智能合约，实际上只是一种改不了Bug的合同

不可篡改性是一个双刃剑

一方面增加了合约的公信力，

但另一方面发布后的软件很难进行修改。发现了安全漏洞，很难发布软分叉进行补救，或者阻止调用；哪怕发现了黑客，也很难冻结其账户

如果你是theDAO的用户，怎么补救？

使用重入攻击转走剩余的资金，之后再分给大家

Nothing is irrevocable

不要过度迷信区块链的不可篡改性，代码是死的，人是活的

比如美国宪法还有修正案，以及被推翻的修正案（禁酒令）

Solidity语言设计的问题

安全性问题

有人认为Solidity语言是反自然的。本质上，转账时调用fallback函数，也就给了被转账方再次调用合约的机会。这和正常情况下转账接收方是被动接收相违背，和常识不同。因此有人提议应该使用函数式的语言编写合约

语言表达能力

图灵完备的表达能力是不是太强了？BTC的语言表达能力太差，ETH写的程序又有些危险。能不能找一种适中的语言？

在日常生活中，我们使用自然语言编写合同，自然语言写出的一些合同也有纠纷，难道要写一套合同专用语言吗？实际上的解决方法是在模板的基础上编写合同，常用的智能合约会出现模板，会出现专门编写智能合约的机构.....

相信智能合约终究会走向成熟！

透明性

中心化的公司很多是闭源的，把源代码当作是商业机密。去中心化的公司为了让大家达成共识，往往需要将代码开源（比如更新，查验等等）

开源的好处

1. 增加合约的公信力
2. 开源代码很多人审查，不容易出现漏洞？

Many Eyeball Fallacy

虽然很多人有权限看，但是大多人没动力看，看不懂或者看懂了找不出漏洞

涉及到财产安全的合约，要仔细检查漏洞。投资需谨慎，将资金投入在自己了解擅长的领域，不要孤注一掷天台见！

去中心化

What does decentralization mean?

去中心化不是说一切由代码决定，也要有人的参与。

去中心化不是说制定好的规则一定不能修改，而是规则的修改要按照去中心化的原则来完成。

分叉

存在分叉的选项，是民主的体现，因为在中心化的世界里，你只能接受！

这就像theDAO当时的子基金，或者像还在挖矿的ETC，亦或是创建以太坊的V神，他们愿意构造一个世界，不是少数服从多数，而是只要有想法，就可以走出自己的道路。

去中心化不等价于分布式

去中心化一定是分布式的，但是分布式如果是由一个组织管辖（或者占有绝对领导权），也不能叫去中心化。

分布式的系统是不同的机器做不同的事情，最后汇总成一个结果。本质目的是为了加速

状态机

而比特币和以太坊都是运行的一台**状态机**，付出巨大的代价使得系统维持一个一致的状态。状态机的本质目的是为了容错。space shuttle, stock exchange 等状态机多台计算机运行同一个状态，防止某台机器宕机的风险。状态机由于同步的需要，往往是机器越多，运行越慢，因此只有几台机器（本来就是防止某台宕机的，多了也没必要）

而向BTC，ETH这种上千台计算机的分布式状态机是两者的混合体，智能合约适用的**不是**分布式系统，智能合约是用来在互不相识的机器之间建立共识机制的控制性语言，而不是用于大规模云计算的。

溢出

Solidity 里的SafeMath有很对针对溢出的函数

```
c=a*b  
assert(c/a==b)
```

不会存在像C语言的溢出